possibly through another GET call. In this example, all the details are available in the listing call, so the second GET is unnecessary, but sometimes, to save bandwidth, the listing doesn't contain all the information.

A button on the home screen allows the user to create a new song. They enter *Booom* by the *Ambassadors*, written in *2119*, making several mistakes along the way. Few APIs implement both PUT and PATCH, and for most mobile applications, only PUT would get implemented. An Edit Song screen already contains all the fields, so why add the extra programming to determine which fields changed; simply use PUT and send the new version of the object to the server. Using the call to PUT (or PATCH), the user fixes their mistakes, correctly setting song 93 to *Boom* (two Os), *X Ambassadors*, and *2019*. Back on the song detail screen, there is a button for deleting a song. This is implemented with the DELETE HTTP method, using the same URL as the song detail and song update calls.

11.1.2 Versioning an API

When writing a Django project, you don't typically have to worry too much about other developers. Even if you're working on a team, the last commit to the repo wins. This can go awry but can usually be handled by good team practices. Adding an API to your project complicates matters. The programmers using your API aren't going to like breaking changes. Drastic modifications to your API may result in someone else's program ceasing to work.

One solution to this problem is API versioning. This is the idea of including a version number in the call, often inside the URL. Developers using your API write their code against a specific version number. You can update your API as long as the change doesn't break the API; for example, adding new fields typically doesn't cause a problem.

When you introduce a breaking change, for example removing an available object or adding a required field, you update the version number. As the number is in the URL, this means a completely new URL. Older versions of the API get maintained for some time, giving developers a chance to catch up.

How long you maintain multiple versions is up to you, but the bigger your audience is, the more time you should give them to change. Many years ago, I was building a site that interacted with Facebook's GraphAPI when they switched from 1.8 to 2.0. They gave 18 months notice before shutting the 1.8 version off completely.

You already have all the knowledge you need to build RESTful APIs in Django. Each API endpoint is just a view. Inside a view, you inspect the HTTP method and perform the requested action. APIs are so common, though, that there are third party libraries to make development simpler for you.

11.2 Building an API with Django Ninja

An API for your project will likely reflect your ORM Model classes. The data you expose through an API is typically the same data in your database but translated into another format. Most API endpoint views query an object out of the database, optionally